

Chain Poker Genesis by LAEV

Multi-Domain Game Protocol

Presentation, Features, and Core Strengths Document

1. Vision Overview

Chain Poker Genesis by LAEV is a game protocol designed as a modular technological infrastructure capable of executing multiple game and betting domains on a unified architectural foundation.

The system was initially designed for **poker**, but its structure was conceived from the outset to support progressive expansion into **sports betting** and **lotteries**, without requiring any redesign of the core system.

The central idea of the protocol is simple yet powerful:

a shared infrastructure, multiple rule engines, and deterministic execution per domain.

2. Purpose of the Protocol

The purpose of Chain Poker Genesis is to provide a technological foundation for rule-based games and markets where every operation can be processed, audited, and verified within a consistent logical framework.

The protocol aims to:

- standardize game execution through state machines,
 - separate infrastructure from rule engines,
 - enable future expansion into new domains,
 - maintain full traceability of every action,
 - provide an architecture prepared for scalability and evolution.
-

3. Starting Point: Poker

The first domain of the protocol is **poker**, as it represents an ideal environment to validate the core architecture.

Poker allows demonstration of:

- well-defined state management,
- strict sequential actions,
- phase-based rule validation,
- determinism requirements,
- execution integrity control.

In this initial stage, Chain Poker Genesis functions as a specialized poker execution engine, consolidating the execution model, event structure, and protocol behavior.

4. Planned Expansion

Once the core is validated with poker, the same protocol expands into other domains through independent rule engines.

Sports Betting

The protocol can be adapted to sports events through:

- creation of events or matches,
- opening and closing of betting markets,
- bet placement,
- result validation,
- automated settlement.

Lottery

The protocol can also incorporate a lottery engine through:

- draw creation,
- ticket issuance,
- sales closure,
- draw execution,
- result validation,
- prize distribution.

In both cases, the underlying infrastructure remains unchanged, and only the domain-specific logic is replaced.

5. Protocol Characteristics

a) Modularity

Each domain operates as an independent rule engine without interfering with others.

b) Determinism

Execution is based on predictable functions and transitions, ensuring consistency and reproducibility.

c) Separation of Concerns

The infrastructure does not define business logic. It only provides execution, logging, validation, and synchronization.

d) Traceability

Every event and transition can be recorded for auditing, analysis, and reconstruction.

e) Domain-based scalability

The protocol grows through modules, adding new games or markets without redesigning the core system.

f) Technological reuse

The same infrastructure serves multiple categories of games, reducing development cost and time.

g) Extensibility

The architecture supports the addition of new rule engines as long as they comply with the protocol's technical contract.

h) Evolution-ready architecture

The system is not limited to a single use case but designed as a multi-domain platform.

6. Core Strengths of the Protocol

1. Structural clarity

Each domain has strict boundaries, preventing business logic from contaminating infrastructure.

2. Operational consistency

The protocol ensures that all nodes or instances process the same rules under the same conditions.

3. Execution integrity

Event-driven design and controlled state transitions ensure unambiguous operational logic.

4. Strategic reuse

A single technological base can support multiple products or markets.

5. Structured growth

Instead of isolated systems, the protocol enables an organized and controlled expansion path.

6. Long-term value

The same infrastructure can evolve into an extensible platform for new gaming and rule-based market verticals.

7. Evolution Model

The protocol follows a staged growth model:

Phase 1: Poker

Validation of the core engine and base infrastructure.

Phase 2: Sports Betting

Expansion into event-based market systems.

Phase 3: Lottery

Integration of draw-based systems and prize validation mechanisms.

Phase 4: Future Engines

Ability to add new domains without modifying the core protocol.

This model allows Chain Poker Genesis to evolve from a specific use case into a broader and more versatile infrastructure platform.

8. Protocol Positioning

Chain Poker Genesis by LAEV should not be understood solely as a poker platform. It should be understood as a **modular game protocol** capable of hosting multiple rule engines on a shared technical infrastructure.

This positions it as a platform-level system rather than a single-product solution.

9. Conclusion

Chain Poker Genesis by LAEV represents an architecture designed for structured game execution, modular expansion, and reusable technological foundations.

Its value lies not only in its initial poker implementation, but in its ability to extend into sports betting, lottery systems, and other rule-based domains that share a common requirement: ordered execution, clear rules, traceability, and operational integrity.

In summary, the protocol proposes a long-term vision:

start with poker, consolidate the infrastructure, and expand into additional game domains through independent rule engines built on a unified technological base.

Chain Poker Genesis by LAEV

Preconfigured Universal State Machine Architecture

Technical Design Document

1. Objective

This document defines how **Chain Poker Genesis by LAEV** achieves multi-domain execution (Poker, Sports Betting, Lottery) through a **single preconfigured state machine design**, eliminating the need to modify or extend the state machine in future phases.

The core principle is:

The state machine is built once, generalized at the lowest possible level, and remains stable for all future rule engines.

Only **domain rule engines (F_domain)** evolve over time — never the state machine core.

2. Core Architectural Principle

The system is based on a strict separation:

Fixed Layer (Never Changes)

- State Machine Runtime
- Event Processing System
- Deterministic Execution Engine
- Consensus Layer
- Persistence Layer (Ledger)

Variable Layer (Extensible)

- Poker Rules Engine
 - Sports Betting Rules Engine
 - Lottery Rules Engine
 - Future Domain Engines
-

3. Key Design Strategy

Instead of designing state machines per game, the system uses:

✓ One Universal Preconfigured State Machine

That state machine is:

- domain-agnostic
 - structurally fixed
 - behavior-driven via external functions
 - pre-designed to support all possible transitions
-

4. Universal State Model

The state machine is defined once using a **generic state schema**:

State =
{

```
state_id,  
domain_id,  
ruleset_id,  
status,  
context,  
payload,  
version,  
flags,  
lifecycle_stage  
}
```

Key Insight:

- The state does NOT encode game logic
 - The state only represents execution conditions
 - Domain logic is externalized
-

5. Preconfigured State Lifecycle (Universal)

Instead of game-specific states, the system uses a **fixed lifecycle skeleton**:

```
CREATED  
↓  
ACTIVE  
↓  
LOCKED  
↓  
RESOLVED  
↓  
FINALIZED
```

Important:

This lifecycle never changes.

All games (Poker, Sports, Lottery) map their internal logic into these stages.

6. Domain Mapping Layer

Each domain engine translates its logic into the universal lifecycle.

Example Mapping

Poker Domain

- CREATED → table created
- ACTIVE → hand in progress
- LOCKED → betting round closed
- RESOLVED → winner determined
- FINALIZED → hand archived

Sports Betting Domain

- CREATED → match registered
- ACTIVE → market open
- LOCKED → market closed
- RESOLVED → result confirmed
- FINALIZED → settlement completed

Lottery Domain

- CREATED → draw created
 - ACTIVE → ticket sales open
 - LOCKED → sales closed
 - RESOLVED → draw executed
 - FINALIZED → prizes distributed
-

7. State Machine Core (Immutable Runtime)

The state machine is defined as:

$\text{State}(n+1) = \text{Transition}(\text{State}(n), \text{Event}, F_domain)$

But the key constraint is:

The Transition function is fixed and never modified.

8. Execution Pipeline

Event Input



Domain Selection (ruleset_id)



Validate Event



Apply Event (generic transformation)



Execute F_domain logic



Map result into universal lifecycle



Persist state



Broadcast consensus

9. Why the State Machine Never Needs Updates

The system is designed so that:

1. All future complexity is absorbed by F_domain

New games = new rule functions, not new states.

2. Lifecycle is abstract, not descriptive

The machine does not understand:

- poker
- betting
- lottery

It only understands:

progression of generic states

3. Events carry all specificity

All domain complexity is encoded in:

- event type
- event payload
- domain_id

Not in the state machine.

10. Domain Engines (Plug-in Model)

Each engine implements:

```
F_domain =  
{  
    validate(state, event),  
    transform(state, event),  
    resolve(state)  
}
```

Constraints:

- Must be deterministic
 - Must not modify runtime
 - Must not extend state machine
 - Must map output to universal lifecycle
-

11. Execution Logic (Core Algorithm)

function EXECUTE(state, event):

```
    domain = SELECT_DOMAIN(event.domain_id)  
  
    if domain.validate(state, event) == false:  
        return ERROR  
  
    state = APPLY_GENERIC_TRANSITION(state, event)  
  
    state = domain.transform(state, event)  
  
    state.lifecycle_stage = MAP_TO_UNIVERSAL_STAGE(state, domain)  
  
    PERSIST(state)  
  
    BROADCAST(state)  
  
    return state
```

12. System Guarantees

This architecture ensures:

✓ **No future modification of state machine**

All evolution happens in domain engines only.

✓ **Full extensibility**

New domains = new F_domain modules only.

✓ **Deterministic execution**

Same input always produces same state.

✓ **Cross-domain consistency**

All systems share identical lifecycle structure.

✓ **Infrastructure stability**

Core runtime remains unchanged indefinitely.

13. Strategic Outcome

This design creates a system that behaves like:

a fixed universal execution engine capable of running infinitely extensible game and market logic without ever modifying its core state machine.

14. Final Conclusion

Chain Poker Genesis by LAEV achieves multi-domain scalability not by evolving the state machine, but by:

- freezing the state machine design permanently,
- abstracting lifecycle into universal stages,
- delegating all logic to external deterministic functions,
- and ensuring that all future systems conform to a single execution model.

This results in a stable core architecture with unlimited domain extensibility.

END OF DOCUMENT